

Trabalho Final de Computação Gráfica

Lucas Vilarinho C. M. Camarço, João Pedro Saleh de Sousa

Departamento de Ciência da Computação - Universidade Federal do Piauí

Teresina, PI, Brasil

Resumo. *Este trabalho consiste no relatório acerca do desenvolvimento do jogo **Zombie Barricade Shooter**, um **First-Person-Shooter** com movimentação fixa desenvolvido em **OpenGL** com uso da **FreeGlut** na linguagem **C++**.*

1. Introdução

O desenvolvimento de videogames expressa a capacidade do desenvolvedor de exprimir ideias a um usuário jogador da melhor forma possível utilizando os recursos dados pelas ferramentas empregadas. Nesse contexto, cabe mencionar o papel da API (Interface de Programação de Aplicativos) gráfica, que é possibilitar a materialização de projetos visuais em 3D ou 2D por intermédio de linguagens de programação já existentes. Em pauta, está a criação de um jogo com a ferramenta OpenGL, aplicada na linguagem C++, como objeto de estudo do campo da computação gráfica em geral.

Portanto, dadas as limitações e, ao mesmo tempo, possibilidades advindas do uso da ferramenta supracitada, foi escolhido como tema da aplicação um *First-Person-Shooter* (jogo de primeira pessoa baseado na funcionalidade de tiros) estacionário, em que o jogador deve permanecer atrás de uma resiliente, mas não invencível, barricada, que cerca sua base de sobrevivência. Sob a proteção dessa barreira, o jogador deve defender-se de criaturas da noite, os mortos-vivos, que surgem da sombria névoa que polui a floresta em que está localizado e almejam destruir seu lar.

A problemática do jogo se dá no seguinte: cada morto-vivo é eliminado somente mediante um disparo certo no crânio, admitido que o número de munições contidas num pente da arma dada ao jogador é limitado. Assim, uma vez que a escassa munição disponível é esgotada, resta ao jogador virar-se, perdendo vista dos inimigos momentaneamente enquanto eles seguem avançando, para recarregar seu armamento. Ademais, os invasores possuem diferentes características, exigindo atenção e boas reações do sobrevivente para cada “tipo” de adversário.

Portanto, o jogo visa, do lado do desenvolvedor, testar as competências acerca da modelagem e texturização de objetos utilizando formas geométricas e funções nativas OpenGL, assim como da integração de bibliotecas externas, do pensamento criativo acerca da qualidade do material e da jogabilidade.

2. Materiais

Durante a produção, foram utilizados diversos recursos (e.g. bibliotecas, arquivos de imagem externos, arquivos *headers*) para possibilitar a concretização de conceitos necessários para a completude da experiência, como sons, modelos geométricos externos e internos ao ambiente, carregamento de imagens e texturas e implementação de animações, entre outros.

2.1 Tiny Obj Loader (Header)

Devido à importância do uso de modelos nesse projeto, foi de bom alvitre a implementação de modelos externos para ilustrar conceitos mais complexos que aprofundassem a ambientação do jogo. A biblioteca em voga faz o carregamento de modelos 3D simples .obj, que são a melhor opção para carregamento leve e sem grandes ambições no quesito de animações ou profundidade. À exemplo, modelos externos presentes no jogo são os modelos da arma, do acampamento e das caixas de munição. Interessantemente, é utilizado um arquivo .mtl em parceria com o carregamento de modelos .obj, que é responsável por informar quais texturas, materiais e cores serão aplicadas em cada célula do modelo, assim como o caminho da textura no projeto.

2.2 STB Image Loader (Header)

O carregamento de arquivos de imagem .png e .jpeg mostrou-se extremamente necessário no complemento dos modelos carregados pela biblioteca citada anteriormente, tanto para que os objetos tridimensionais fossem “pintados” quanto para o processamento de suas normais, que também constavam num arquivo de imagem separado. Assim, é com o uso dessa biblioteca que foram carregados no ambiente as texturas utilizadas pelo modelo das caixas de munição, pelo acampamento do jogador, pela metralhadora e pela barricada.

2.3 MiniAudio (Header)

A reprodução de arquivos de áudio em aplicações OpenGL utilizando C++ é trabalhosa. Entretanto, existem bibliotecas *headers* que possibilitam o uso desse recurso, e dentre essas bibliotecas, a MiniAudio é uma das mais versáteis devido à possibilidade de playback de diversos arquivos concomitantemente e à sua fácil instalação, bastando o descarregamento do arquivo cabeçalho na pasta raiz junto aos outros arquivos programas C++. No projeto em pauta, a mesma foi usada na reprodução de sons ambiente, sons de disparos de armas de fogo diversos, entre outros.

2.4 Bibliotecas auxiliares externas

Foram aplicadas funções de bibliotecas típicas de programas em C++ como <iostream>, para facilitar a escrita de outputs e inputs, <cmath>, para o cálculo e aplicação de funções matemáticas mais complexas dependentes de propriedades como seno ou cosseno para animações e movimentos, <cstdlib>, para facilitação de conversão de tipos e capacidades de propósito geral na linguagem, <ctime>, para propriedades relativas e dependentes da contagem de tempo, <string>, para escrita de caracteres *in-game*, <vector>, para melhor uso de vetores de elementos variados, e <map>, para associação de elementos numa relação chaveada e/ou mapeada.

3. Métodos

Para a realização da visão do jogo, foi necessária a implementação de diversas técnicas e funcionalidades comuns no contexto de *game-making*.

3.1 Movimentação

Enquanto o jogador permanece, em termos de movimento, limitado aos estados de olhar para a barricada ou olhar para suas caixas de munição/acampamento, os mortos-vivos vêm, da sombria floresta, constantemente avançando rumo ao jogador. Isso se dá por meio do incremento da coordenada Z de cada zumbi, carregando consigo a estrutura visível e invisível do objeto, como seu modelo e sua hitbox. A título de curiosidade, a ilusão de movimento quando o jogador vira a câmera para recarregar a arma é feita por meio de cálculos de interpolação e rotação entre os pontos em torno de um eixo.

3.2 Colisão e raycasting

A supramencionada estrutura invisível do objeto zumbi (sua hitbox) pode ser resumida a um formato cúbico do tamanho quase exato da cabeça visível do modelo. O jogo implementa a mecânica de tiro utilizando Raycasting para detectar acertos nessa hitbox, resultando em um headshot. Quando o jogador atira, um raio com origem e direção é lançado a partir da câmera para onde está o mouse do jogador, utilizando as funções de projeção do OpenGL. Para cada zumbi, o código define uma AABB (Axis-Aligned Bounding Box) que envolve a região da cabeça, e então utiliza o algoritmo de interseção Ray vs AABB ("Slab") para verificar se o raio atinge essa caixa. Se um acerto for detectado, o objeto zumbi é removido e um efeito de explosão de sangue é acionado, simulando a eliminação do zumbi. Também, essa hitbox é constantemente checada para ratificar se o zumbi está em contato com a colisão da barricada, decidindo se o mesmo está em modo de ataque ou não.

3.3 Modelos e texturas

Modelos mais complexos, como o acampamento, as caixas de munição e a arma de fogo tiveram que ser carregados em formato .obj, junto com suas texturas em imagens e seus guias em formato .mtl. Porém, modelos mais simples como as árvores, os zumbis e a barricada foram feitas em código, usando as primitivas da API gráfica como cubóides, cones e cilindros. É relevante também mencionar que certos objetos que usam primitivas do OpenGL, como os zumbis e as árvores, são coloridos em código.

3.4 Iluminação e visibilidade

O ambiente inteiro é iluminado por uma simples iluminação global de tom azul escuro para dar a ilusão de um ambiente noturno e sombrio, com uma luz em ponto (spotlight) acima do jogador para lhe dar maior visibilidade. Ademais, é aplicada uma forte névoa (fog) a partir de uma certa distância do jogador, tornando gradual o surgimento dos zumbis. Outra dinâmica interessante é a de *muzzle-flash*, que se refere ao flash de luz vindo do cano da arma mediante um disparo efetuado, efeito esse que é adquirido fazendo um aumento do valor de brilho dos pixels próximos à posição do cano da arma na tela.

3.5 Comportamentos e inteligência

A estrutura de um zumbi possui uma variável de tipo, que é decidido aleatoriamente toda vez que uma nova estrutura de zumbi é gerada, que pode variar entre 3 alternativas: runners (maior velocidade rumo à barricada), straffers (variam sua posição lateral em movimentos harmônicos) e normals (caminham lentamente rumo à barricada em linha reta).

3.6 Animações

Para a animação de disparo efetuado e de recarregamento, há um deslocamento e rotação do modelo carregado da arma, dando ilusão de deslocamento devido ao coice e de aproximação das caixas de munição. Além disso, há um controle dos membros individuais do zumbi (cubóides que representam seus braços e pernas) para suas animações de ataque ou de caminhada usando funções de tempo, ângulos e propriedades matemáticas, dependendo do seu estado de ação. Há, também, uso de um pequeno sistema de partículas vermelhas para emular a explosão do crânio de um zumbi sempre que um disparo é bem-sucedido.

4. Resultados e discussão

O *game-loop* resume-se à boa administração de munição e de tempo: o jogador deve, idealmente, evitar ficar sobrecarregado a ponto de que falte munição contra os zumbis que estão a caminho. Sobreviver o maior tempo possível deve ser a prioridade.



Figura 1. Ambientação do jogo antes da chegada dos inimigos.

Na figura 1 é retratado o ambiente inicial do jogo, antes do avanço das criaturas. Acima, percebem-se as métricas importantes ao jogo, que é a integridade da barricada e o número de balas ainda disponíveis para disparo.



Figura 2. Zumbis vindo rumo ao jogador e atacando sua barricada, que dispara em retaliação.

Na figura 2, é notável o ataque incessante dos zumbis à barricada, além do recurso visual do flash de luz advindo do cano da arma durante um disparo efetuado.



Figura 3. Jogador virado de costas para a barricada, recarregando sua arma.

Na figura 3, observa-se que, gradativamente e passivamente, as balas do jogador são recarregadas quando ele está virado para as caixas de munição. Entretanto, nesse meio tempo, os zumbis continuam surgindo e avançando. O contador acima (balas) é incrementado.

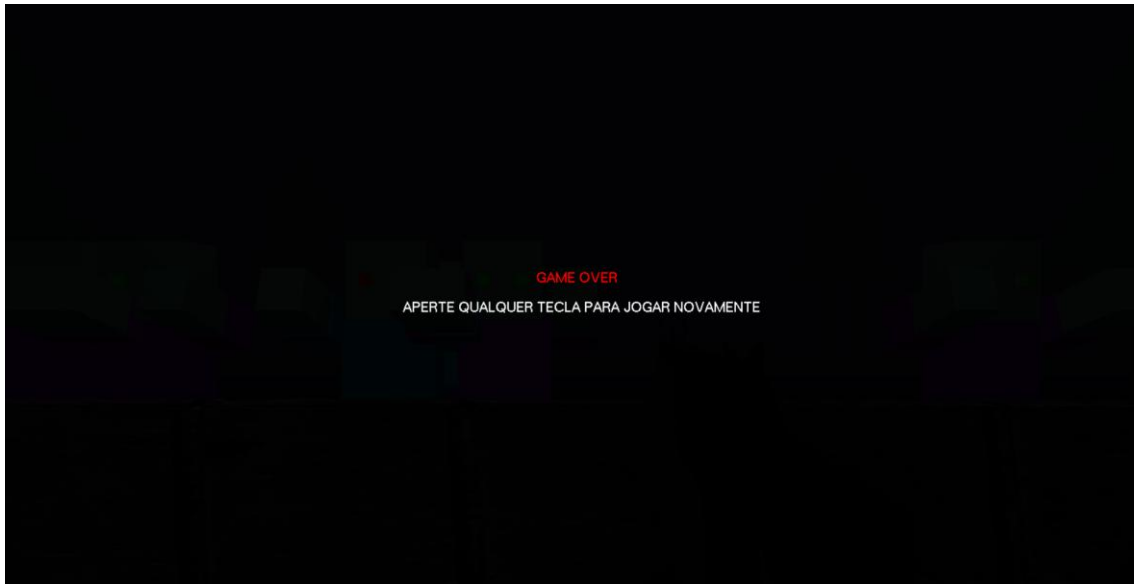


Figura 4. Tela de GAME OVER quando zumbis derrubam a barricada.

Vê-se que, na figura 4, uma vez que a “vida” da barricada é esgotada, o jogador perde o jogo, sendo atacado pelos zumbis. O tempo do jogo é parado, o jogador perde os controles, e a tela mostrada é exibida, dando a opção do jogador de recomeçar enquanto ouve “scream.mp3” ser tocado, enfatizando o fim do personagem naquela tentativa.

5. Conclusão

O desenvolvimento do Zombie Barricade Shooter cumpriu com êxito o objetivo de aplicar conceitos de Computação Gráfica na criação de um First-Person Shooter estacionário utilizando a API OpenGL e a linguagem C++. O projeto permitiu testar e validar competências essenciais, como modelagem, texturização e implementação de lógica de jogabilidade. Dentre os desafios enfrentados, destacou-se a complexidade de gerenciar recursos multimídia (áudio e modelos 3D) em um ambiente C++ nativo, uma tarefa trabalhosa. As soluções adotadas para mitigar essa dificuldade envolveram a integração de bibliotecas externas específicas. Além disso, a implementação da mecânica de tiro exigiu precisão matemática, sendo solucionada através da técnica de Raycasting e do algoritmo de interseção Ray vs AABB, garantindo a detecção correta de headshots.

5. Referências

FOSNER, M. OpenGL Utility Toolkit (GLUT) 3.7. Disponível em: <http://www.opengl.org/resources/libraries/glut/>. Acesso em: 26 nov. 2025.

YOKOGAWA, S. tinyobjloader 2.0. Disponível em: <https://github.com/tinyobjloader/tinyobjloader>. Acesso em: 26 nov. 2025.

GARRETT, N. stb_image 2.27. Disponível em: <https://github.com/nothings/stb>. Acesso em: 26 nov. 2025.

DUMONT, D. miniaudio. Disponível em: <https://miniaud.io/>. Acesso em: 26 nov. 2025.